EG Invoice Assistant Documentation

Title: EG Invoice Assistant Documentation
Date: 09-May-2025
Prepared by: Nithin Nayak
Reviewed by: Prakhyath Rai, Suraj Nagraj
Version: v1.0

Contents

Section No.	Section Title	Page Number
1.	Objective	3
2.	Overview	3
3.	Theory	3
4.	Architecture	5
5.	Workflow	7
6.	Cost Analysis	8
7.	Functional Requirements	7
8.	Non-Functional Requirements	8
9.	Enhancements	9
10.	Security Considerations	10
11.	Result and Analysis	10
12.	Conclusion	12

1. Objective

The objective of this document is to detail the design, architecture, and functionality of the EG Invoice Assistant system. This system aims to enhance the classification and extraction of financial documents such as invoices and receipts using a Large Language Model (LLM) via AWS Bedrock. By extracting key details like vendor information, amounts (with and without tax), and expense descriptions, the tool empowers finance teams and managers to efficiently process and analyse financial documents, ultimately improving accuracy and productivity.

2. Overview

The EG Invoice Assistant is designed to optimize the classification and extraction process of financial documents by automating the identification and data extraction from invoices and receipts. Utilizing a Large Language Model (LLM) via AWS Bedrock, the system extracts key details such as vendor information, amounts (with and without tax), and expense descriptions, helping finance teams and managers to efficiently process and analyze financial documents.

3. Theory

3.1. FastAPI Backend

FastAPI is a modern, high-performance web framework for building APIs with Python. It is known for its speed and ease of use. In this system:

- It handles incoming API requests from the frontend.
- Validates uploaded documents and metadata.
- Coordinates with other services like AWS S3, Bedrock, and MongoDB.
- Returns structured responses to the user.

3.2. AWS Route 53

AWS Route 53 is a scalable and highly available Domain Name System (DNS) web service. It:

- Translates human-readable domain names (like invoiceassistant.com) into IP addresses.
- Routes user requests to the correct AWS resources, such as the Application Load Balancer (ALB).

3.3. Application Load Balancer (ALB)

The ALB is a part of AWS Elastic Load Balancing. It:

- Distributes incoming application traffic across multiple targets (like containers or EC2 instances).
- Ensures high availability and fault tolerance.
- Supports HTTPS for secure communication.

3.4. AWS Fargate

AWS Fargate is a serverless compute engine for containers. It:

- Runs containerized applications without needing to manage servers.
- Automatically scales based on demand.
- Hosts the FastAPI backend in isolated, secure environments.

3.5. Amazon S3 (Simple Storage Service)

Amazon S3 is a cloud-based object storage service. It:

- Stores uploaded invoice and receipt documents.
- Offers high durability, availability, and security.
- Allows controlled access via permissions and policies.

3.6. AWS Bedrock

AWS Bedrock provides access to foundation models from various AI providers. In this system:

- It uses a Large Language Model (Claude) to analyze documents.
- Extracts structured data like vendor names, amounts, and descriptions from invoices.
- Returns the extracted data to the backend for further processing.

3.7. MongoDB

MongoDB is a NoSQL database known for its flexibility and scalability. It:

- Stores the extracted invoice data in JSON-like documents.
- Supports fast read/write operations.
- Is ideal for handling semi-structured data like the output from LLMs.

3.8. Keycloak

Keycloak is an open-source identity and access management solution. It:

- Manages user authentication (login/logout).
- Supports role-based access control.
- Integrates with various identity providers (e.g., LDAP, OAuth2, SAML).

4. Architecture

The EG Invoice Assistant system is composed of the following core components:

a. **Frontend**: A React-based web application deployed as a containerized service on Amazon ECS using AWS Fargate for serverless compute.

b. **Backend**: A FastAPI service, also containerized and deployed on ECS with Fargate, responsible for handling API requests and orchestrating processing workflows.

c. **DNS Management**: Domain resolution is managed via **Amazon Route 53**, providing high availability and scalability.

d. **Load Balancer**: An **Application Load Balancer (ALB)** distributes incoming traffic across frontend and backend services, ensuring availability and performance.

e. **Container Orchestration**: **AWS Fargate** is used to run containers without managing servers, simplifying scaling and maintenance.

f. **Document Storage**: **Amazon S3** serves as the primary storage for uploaded and processed invoice documents.

g. **LLM-Based Processing**: **Amazon Bedrock** integrates large language model capabilities to extract and process information from invoice documents.

h. **Database**: **MongoDB** is used to store structured data extracted from documents, supporting flexible and scalable schema design.

i. **Authentication & Authorization**: **Keycloak** handles identity management, including user login, role-based access control, and token issuance.

The architecture diagram is as shown below in figure 4.

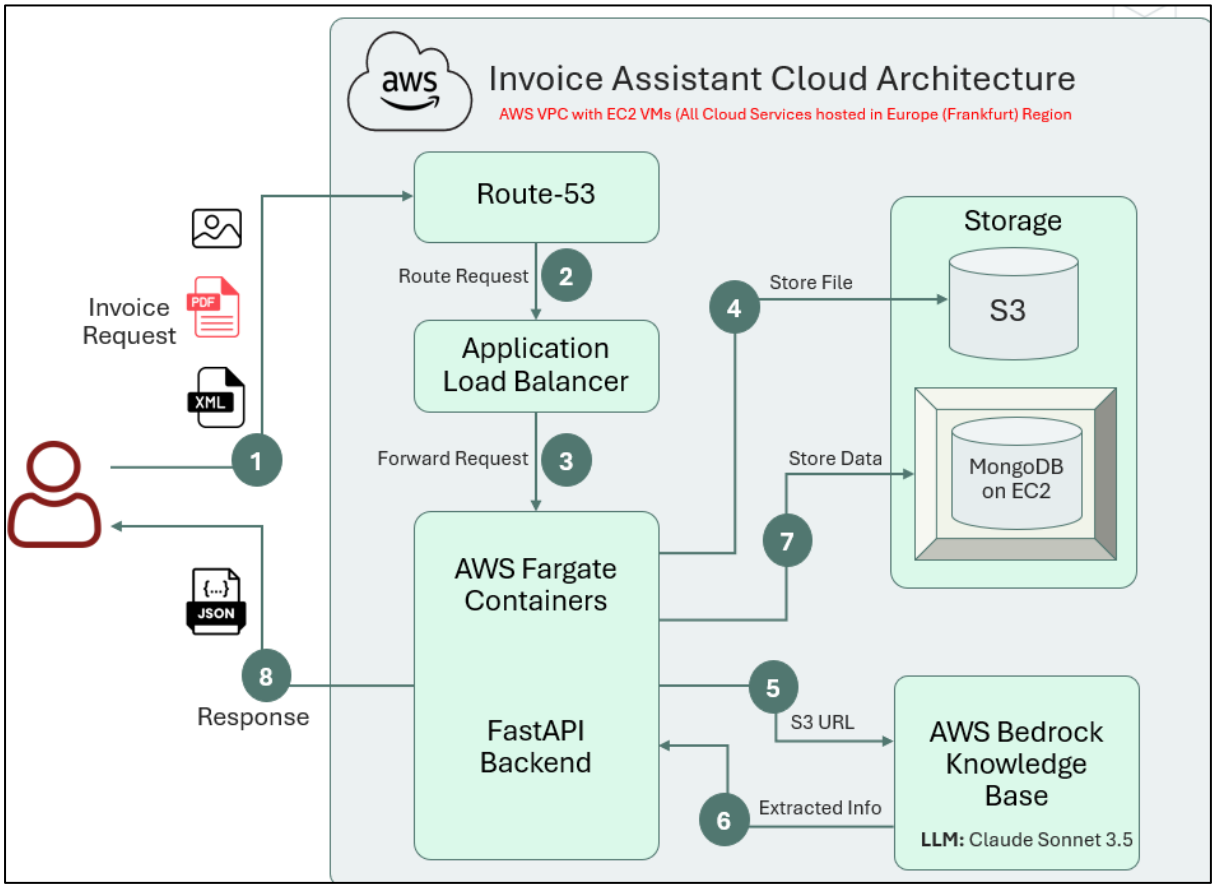

Figure 4: Architecture Diagram

5. Workflow

The workflow of the EG Invoice Assistant web application is as follows:

a. The user uploads the document and sends a request containing meta information and an invoice document ID to the processing API.

b. AWS Route 53 resolves the DNS to the Application Load Balancer (ALB).

c. The ALB forwards the request to the FastAPI backend hosted on AWS Fargate containers.

d. The FastAPI backend validates the document and stores it in an Amazon S3 bucket.

e. FastAPI sends a request to the Bedrock service, providing the S3 URL of the stored invoice document.

f. Bedrock processes the invoice and extracts key details such as amounts, dates, vendor names, and line items.

g. FastAPI receives the extracted information, performs additional processing or validation, and stores it in MongoDB.

h. FastAPI prepares a JSON response containing the extracted information and a confidence score, and returns it to the user.

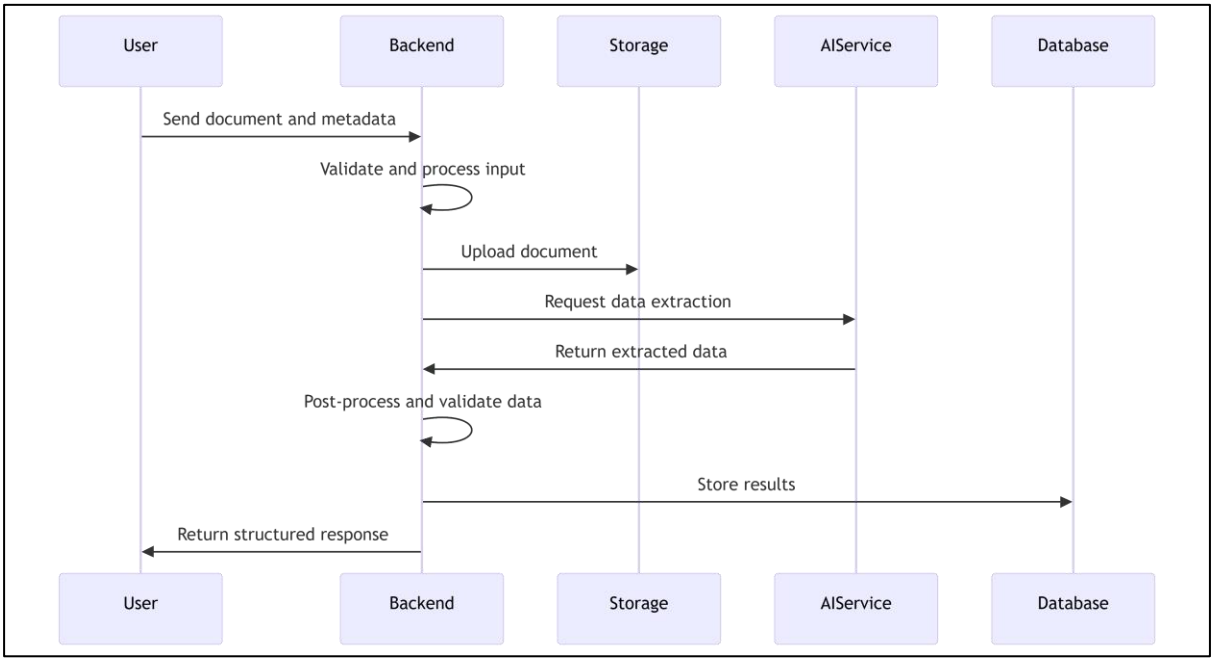

Figure 5 a: Sequence Diagram

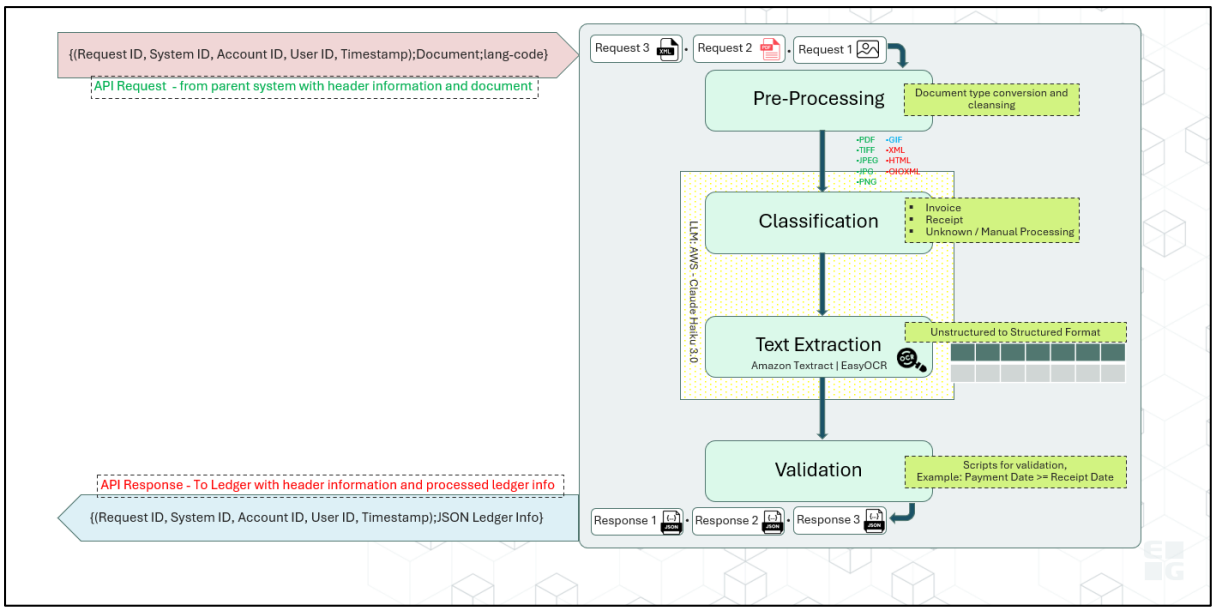

Figure 5 b: Flow Diagram

The flow of the application is as shown in Figure 5a and 5b respectively.

6. Cost Analysis

To process a single invoice document, the system incurs costs based on three primary components: token usage and processing time. Specifically, the input token size for a document is approximately 3000 tokens, which costs **$0.0024**. The response generated by the AI model typically consists of around 1000 tokens, costing **$0.04**. Additionally, the average time taken to process one document is about 10 seconds, which translates to a compute cost of **$0.000701861**. These values collectively represent the operational cost per document and help in estimating the overall expense for large-scale invoice processing.

Assumptions on Document Size for Cost Derivation		
Ticket Details	**Unit**	**Cost (in $)**
Input Tokens Size for a Document	3000	0.0024
Response Token Size	1000	0.04
Time to process 1 Documents (in Seconds)	10	0.000701861
Total	**0.043101861**	

1 User		
Number Of Documents	Approx Run Time (seconds)	Cost ($)
1	10	160
10	100	160
25	250	161
50	500	162
100	1000	164
500	5000	181
1000	10000	203

7. Functional Requirements

a. The system should allow users to upload financial documents of various formats.

b. The system should classify the uploaded documents into invoices and receipts.

c. The system should extract key details from the documents such as vendor details, amount with tax, amount without tax, and description of expense type.

d. The system should store the uploaded documents in Amazon S3.

e. The system should store the extracted information in MongoDB.

f. The system should return the extracted information and a confidence score to the user.

8. Non-Functional Requirements

a. The system should be scalable to handle many document uploads and processing requests.

b. The system should be secure to protect sensitive financial information.

c. The system should be reliable to ensure high availability and durability of stored documents and extracted information.

d. The system should have low latency to provide quick responses to user requests.

9. Enhancements

a. Implement additional document types for classification and extraction.

b. Improve the accuracy of the LLM used for extraction.

c. Add support for dynamic prompting.

d. Integrate with other financial systems for seamless data exchange.

10. Security Considerations

a. Use HTTPS for secure communication between the frontend and backend.

b. Implement access control permissions for Amazon S3 buckets to restrict access to authorized services.

c. Use Keycloak for user authentication and authorization.

d. Regularly update and patch all components to protect against security vulnerabilities.

11. Result and Analysis

The figure below illustrates several document samples that have been classified as either invoices or receipts. For each document type, relevant data has been extracted, showcasing key fields and values identified within the documents. The extracted data provides valuable insights into the structure and content of each document, enabling better understanding and processing of these document types. The invoice samples are shown in figure 11 a and b respectively and receipt samples in figure 11 c and d respectively.

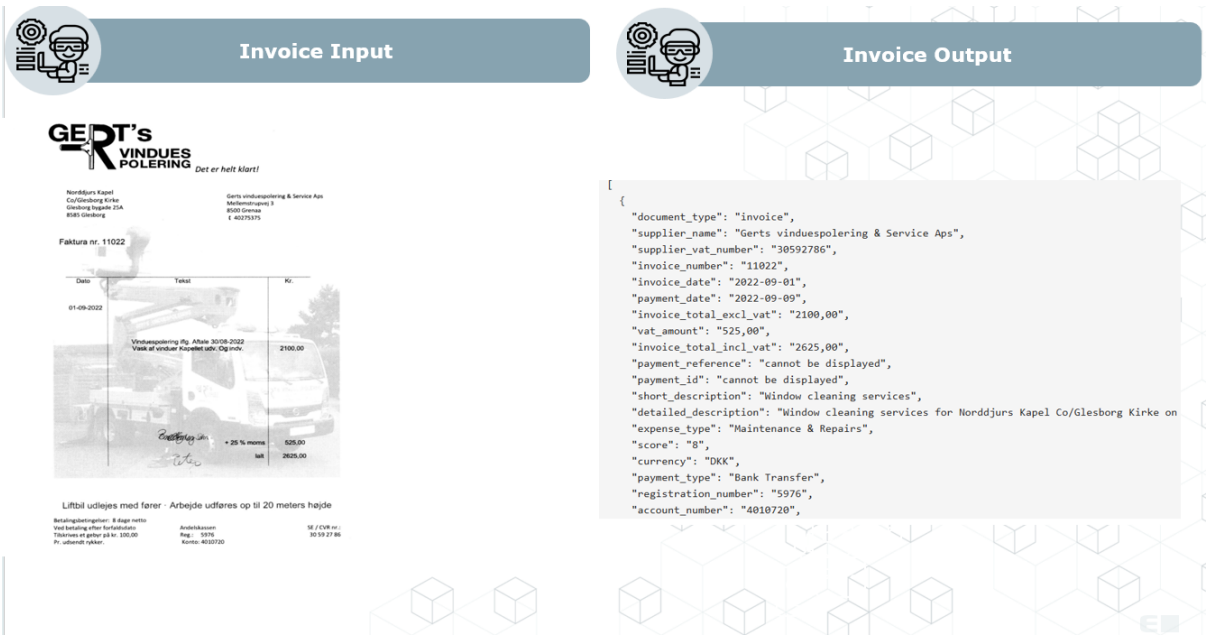

Figure 11 a: Invoice Sample-1

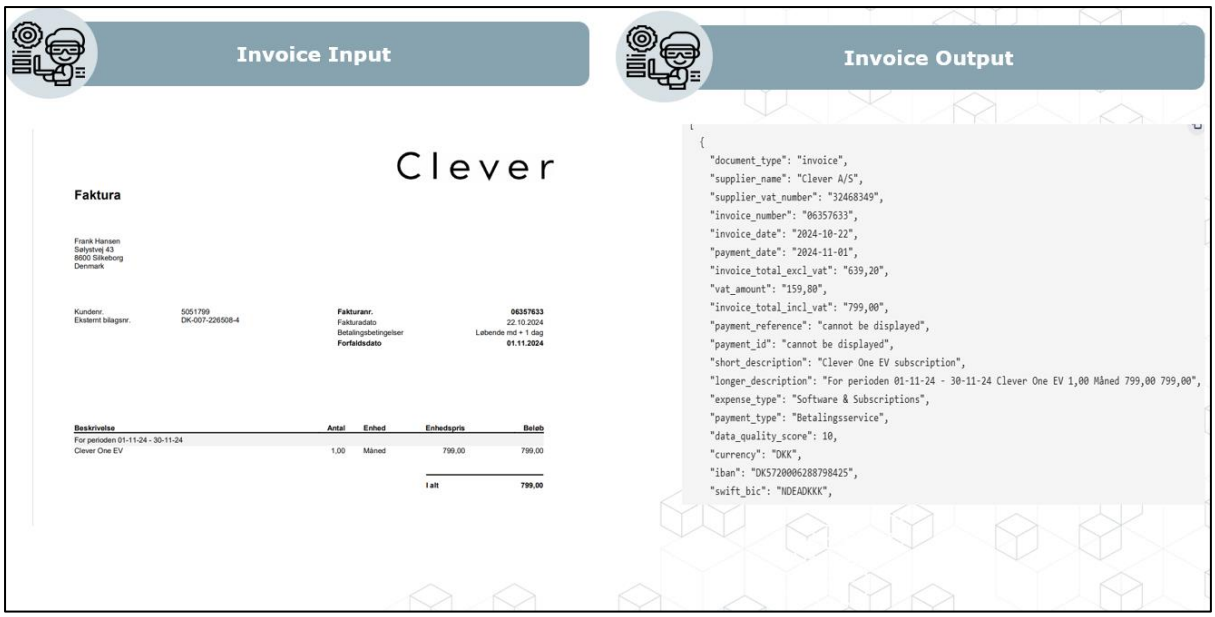

Figure 11 a: Invoice Sample-2

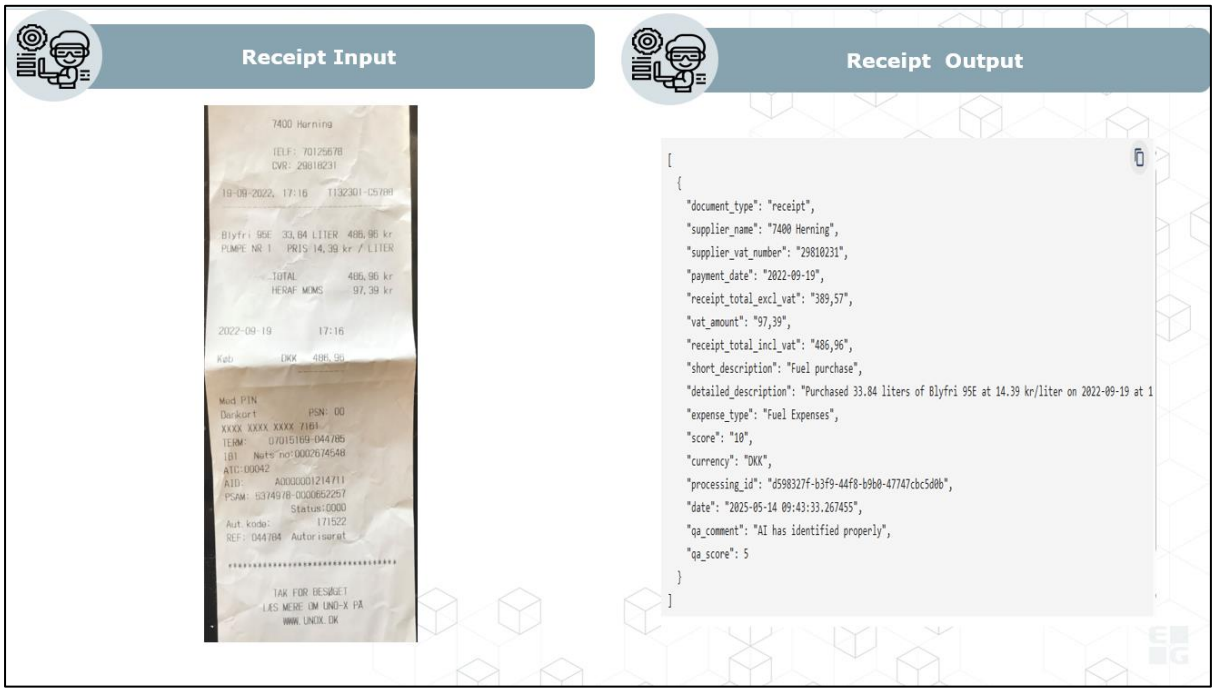

Figure 11 c: Receipt Sample-1

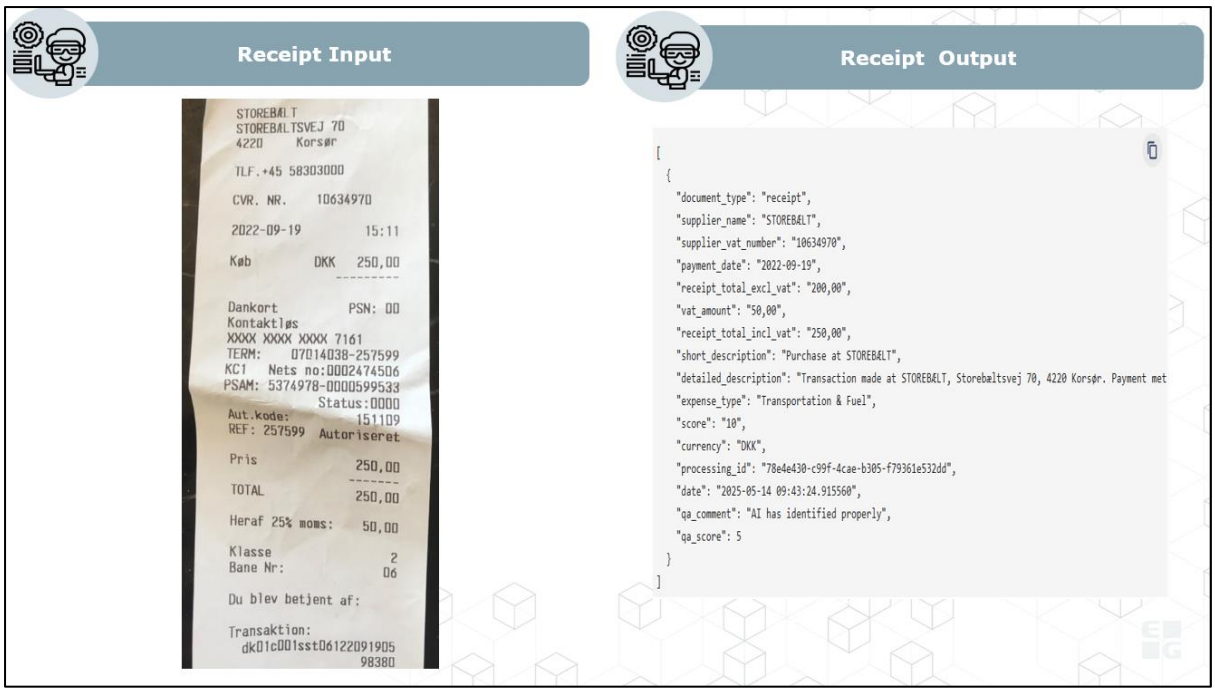

Figure 11 d: Receipt Sample-1

12. Conclusion

The EG Invoice Assistant web application provides a robust solution for classifying financial documents and extracting key details from them. With its scalable architecture, secure storage, and accurate extraction capabilities, it offers a valuable tool for managing financial information efficiently.